# AI Workfinder

Roman Nadkernychnyi
Maksim Pyanin
Bohdan Hrytsai
Maksym Stepaniuk

## 1 Testy

Aplikacja została poddana testom automatycznym obejmującym zarówno warstwę backendową, jak i frontendową. Celem przeprowadzonych testów była weryfikacja poprawności działania kluczowych modułów systemu, sprawdzenie logiki biznesowej, obsługi błędów, poprawności przetwarzania danych wejściowych oraz stabilności interfejsów aplikacyjnych. Testy miały również na celu potwierdzenie bezpieczeństwa wybranych mechanizmów systemu oraz ograniczenie ryzyka występowania błędów regresyjnych w kolejnych etapach rozwoju projektu.

### 1.1 Zakres testów

Zakres testów objął komponenty backendowe i frontendowe projektu. Struktura testów została rozdzielona zgodnie z architekturą aplikacji i rolą poszczególnych modułów.

#### 1.1.1 Struktura testów

Testy backendowe znajdują się w katalogach:

- `backend/tests/unit`,
- `backend/tests/integration`.

W ramach testów backendowych wyróżniono następujące sekcje:

- `unit/core`,
- `unit/repositories`,
- `unit/services`,
- `unit/services/ai`,
- `unit/tools`,
- `integration/api/routes`.

Testy frontendowe znajdują się w katalogu:

- `job-agent-frontend/src/test`.

W warstwie frontendowej wyróżniono następujące sekcje:

- `node/api`,
- `node/smoke`,

- `dom/app`,
- `dom/components`,
- `dom/contexts`,
- `dom/pages`,
- `dom/router`.

### 1.1.2 Liczba testów

Łącznie w repozytorium przygotowano 130 testów automatycznych, z czego:

- 89 testów dotyczy backendu,
- 41 testów dotyczy frontendu.

W obrębie backendu wydzielono:

- `unit/core` - 4 testy,
- `unit/repositories` - 9 testów,
- `unit/services` - 12 testów,
- `unit/services/ai` - 13 testów,
- `unit/tools` - 24 testy,
- `integration/api/routes` - 27 testów.

W obrębie frontendu wyróżniono:

- `node/api` - 10 testów,
- `node/smoke` - 4 testy,
- `dom/app` - 2 testy,
- `dom/components` - 4 testy,
- `dom/contexts` - 5 testów,
- `dom/pages` - 13 testów,
- `dom/router` - 3 testy.

### 1.1.3 Wykorzystane biblioteki i narzędzia

Do testowania warstwy backendowej wykorzystano następujące narzędzia:

- `pytest`,
- `pytest-cov`,
- `httpx`,
- `fastapi.testclient`.

Do testowania warstwy frontendowej wykorzystano:

- `vitest`,
- `@testing-library/react`,
- `@testing-library/jest-dom`,
- `@testing-library/user-event`,
- `happy-dom`,
- `@vitest/coverage-v8`.

### 1.1.4 Zakres przetestowanych przypadków

W warstwie backendowej przetestowano przede wszystkim:

- bezpieczeństwo: haszowanie hasła, weryfikację hasła, obsługę tokenów JWT, przypadki tokenu błędnego i wygasłego,
- repozytorium użytkownika: operacje CRUD, pobieranie po adresie e-mail i identyfikatorze, pusty update, lazy collection oraz indeksowanie,
- profil użytkownika: odczyt danych, aktualizację profilu, upload CV, pusty plik, błędny format, brak nazwy pliku oraz plik większy niż 5 MB,
- mechanizmy autoryzacji: rejestrację, duplikat adresu e-mail, poprawny i błędny login,
- moduł wyszukiwania: walidację danych wejściowych, brak CV, brak pełnej nazwy użytkownika, brak pliku CV, pustą specjalizację, blokadę równoległego wyszukiwania, status, stream oraz ścieżki błędów,
- moduł aplikacji: normalizację statusów oraz filtrowanie tylko wysłanych aplikacji,
- moduły AI i narzędzia pomocnicze: parser, crawler, mechanizm one-click apply, apply HTTP, graph flow, runner oraz CLI.

W warstwie frontendowej przetestowano:

- klient HTTP: auth header, custom base URL, operacje GET/POST/PUT, `FormData`, odpowiedzi plain-text i JSON, obsługę błędów oraz interceptory,
- warstwę API wrapperów: `auth`, `jobs`, `applications`, `gmail`,
- warstwę UI i DOM: logowanie, rejestrację, profil, dashboard, widok aplikacji, listę ofert pracy, protected route, auth context oraz i18n.

### 1.1.5 Wyniki wykonania testów backendu

Potwierdzony wynik wykonania backendowych testów automatycznych był następujący:

- środowisko: Python 3.13.2, pytest 9.0.2,
- wynik: 89 passed, 0 failed, 0 skipped,
- czas wykonania: 9.07 s,
- liczba ostrzeżeń: 532,
- pokrycie kodu: 95%,
- podsumowanie coverage: 1126 statements, 52 missed.

### 1.1.6 Wyniki wykonania testów frontend HTTP/API

Potwierdzony wynik wykonania testów frontendowych dla warstwy HTTP/API przedstawiał się następująco:

- wynik: 14 passed,
- liczba plików testowych zakończonych sukcesem: 6,
- pokrycie dla `src/api/**/*.js`:
  - Lines: 100%,
  - Statements: 100%,
  - Functions: 100%,
  - Branches: 91.3%.

### 1.1.7 Uwagi metodologiczne

Należy zaznaczyć, że backendowe testy integracyjne są głównie testami na poziomie handlerów i tras, a nie pełnymi testami HTTP end-to-end. Warstwa danych nie korzysta podczas testów z rzeczywistej bazy danych, lecz ze stubowanej `FakeCollection` działającej w pamięci. Dodatkowo w projekcie istnieje fixture oparta o `TestClient`, lecz obecnie nie jest ona używana bezpośrednio w testach.

### 1.1.8 Raporty testowe

W projekcie przygotowano następujące raporty:

- backendowy raport HTML z testów: `backend/reports/tests/backend-tests.html`,
- backendowy raport HTML pokrycia kodu: `backend/reports/coverage/index.html`,
- frontendowy raport HTML pokrycia kodu: `job-agent-frontend/coverage/index.html`,
- frontendowe podsumowanie coverage w formacie JSON: `job-agent-frontend/coverage/coverage-`

## 1.2 Kluczowe przypadki testowe

W procesie przygotowywania testów wyodrębniono najważniejsze przypadki testowe obejmujące zarówno logikę backendową, jak i wybrane elementy frontendu. Przypadki te pozwalają zweryfikować poprawność działania aplikacji w scenariuszach standardowych, błędnych oraz brzegowych.

| ID | Opis | Wynik |
|---|---|---|
| UT1 | Haszowanie hasła i poprawna weryfikacja wygenerowanego skrótu. | Sukces |
| UT2 | Odrzucenie błędnego lub wygasłego tokenu JWT. | Sukces |
| UT3 | Utworzenie użytkownika w repozytorium i pobranie po adresie e-mail. | Sukces |
| UT4 | Obsługa pustego update użytkownika bez uszkodzenia danych. | Sukces |
| UT5 | Poprawna obsługa lazy collection i indeksu dla repozytorium użytkownika. | Sukces |
| UT6 | Walidacja przejść stanu procesu wyszukiwania oraz zapisu event log. | Sukces |
| UT7 | Poprawne uruchomienie modułu AI runner i graph flow. | Sukces |
| UT8 | Normalizacja danych wejściowych przez parser. | Sukces |
| UT9 | Obsługa wyjątku w narzędziu apply HTTP. | Sukces |
| UT10 | Test działania crawlera oraz one-click apply. | Sukces |
| IT1 | Rejestracja użytkownika z poprawnymi danymi. | Sukces |
| IT2 | Próba rejestracji z duplikatem adresu e-mail. | Błąd obsłużony poprawnie |
| IT3 | Logowanie z poprawnymi danymi uwierzytelniającymi. | Sukces |
| IT4 | Logowanie z błędnym hasłem. | Błąd autoryzacji obsłużony poprawnie |
| IT5 | Pobranie i aktualizacja profilu użytkownika. | Sukces |
| IT6 | Przesłanie pustego pliku CV. | Błąd walidacyjny obsłużony poprawnie |
| IT7 | Przesłanie pliku CV w nieobsługiwanym formacie. | Błąd walidacyjny obsłużony poprawnie |
| IT8 | Przesłanie pliku większego niż 5 MB. | Błąd walidacyjny obsłużony poprawnie |
| IT9 | Uruchomienie wyszukiwania bez wymaganego CV. | Błąd walidacyjny obsłużony poprawnie |
| IT10 | Zablokowanie równoległego uruchomienia procesu wyszukiwania. | Sukces |
| IT11 | Pobranie statusu i stream dla procesu wyszukiwania. | Sukces |
| FT1 | Dodanie auth header w kliencie HTTP frontendu. | Sukces |
| FT2 | Obsługa odpowiedzi JSON i plain-text w warstwie API. | Sukces |
| FT3 | Test wrapperów API dla modułów auth, jobs, applications i gmail. | Sukces |

| | | |
|---|---|---|
| FT4 | Renderowanie widoków login, register i profile. | Sukces |
| FT5 | Działanie protected route oraz auth context. | Sukces |

Tabela 1: Przykładowe przypadki testowe

## 1.3 Podsumowanie wyników testów automatycznych

### 1.3.1 Statystyki testów backendowych

Tabela 2: Statystyki testów backendowych

| **Sekcja** | **Liczba testów** | **Stan** |
|---|---|---|
| `unit/core` | 4 | 100% sukces |
| `unit/repositories` | 9 | 100% sukces |
| `unit/services` | 12 | 100% sukces |
| `unit/services/ai` | 13 | 100% sukces |
| `unit/tools` | 24 | 100% sukces |
| `integration/api/routes` | 27 | 100% sukces |
| **Razem** | **89** | **100% sukces** |

### 1.3.2 Statystyki testów frontendowych

Tabela 3: Statystyki testów frontendowych

| **Sekcja** | **Liczba testów** | **Stan** |
|---|---|---|
| `node/api` | 10 | przygotowane |
| `node/smoke` | 4 | przygotowane |
| `dom/app` | 2 | przygotowane |
| `dom/components` | 4 | przygotowane |
| `dom/contexts` | 5 | przygotowane |
| `dom/pages` | 13 | przygotowane |
| `dom/router` | 3 | przygotowane |
| **Razem** | **41** | **częściowo potwierdzone** |

### 1.3.3 Statystyki ogólne projektu

Tabela 4: Zbiorcze statystyki testów w projekcie

| **Kategoria** | **Liczba testów** | **Stan** |
|---|---|---|
| Backend | 89 | potwierdzone |
| Frontend | 41 | częściowo potwierdzone |
| **Łącznie** | **130** | **projekt objęty szerokim zakresem testów** |

### 1.3.4 Pokrycie kodu

Backend osiągnął 95% pokrycia kodu przy 1126 analizowanych instrukcjach i 52 niewykonanych. Wynik ten należy uznać za bardzo wysoki i świadczący o szerokim zakresie automatycznej weryfikacji logiki systemu. W przypadku testów frontendowych potwierdzono pełne pokrycie linii, instrukcji i funkcji dla warstwy `src/api/**/*.js`, przy jednoczesnym pokryciu gałęzi na poziomie 91.3%.

### 1.3.5 Frontend

**All files**

**100%** Statements 45/45 **73.91%** Branches 17/23 **100%** Functions 15/15 **100%** Lines 45/45

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

| File | | Statements | | Branches | | Functions | | Lines | |
|---|---|---|---|---|---|---|---|---|---|
| applications.js | | 100% | 1/1 | 100% | 0/0 | 100% | 1/1 | 100% | 1/1 |
| auth.js | | 100% | 4/4 | 100% | 0/0 | 100% | 4/4 | 100% | 4/4 |
| gmail.js | | 100% | 1/1 | 100% | 0/0 | 100% | 1/1 | 100% | 1/1 |
| http.js | | 100% | 36/36 | 73.91% | 17/23 | 100% | 6/6 | 100% | 36/36 |
| jobs.js | | 100% | 3/3 | 100% | 0/0 | 100% | 3/3 | 100% | 3/3 |

Rysunek 1: Pokrycie testów frontendowych

### 1.3.6 Backend

Coverage report: 95%

Files | Functions | Classes

filter...

hide covered

| File ▲ | statements | missing | excluded | coverage |
|---|---|---|---|---|
| backend / app / api / deps.py | 19 | 0 | 0 | 100% |
| backend / app / api / routes / applications.py | 34 | 0 | 0 | 100% |
| backend / app / api / routes / auth.py | 29 | 0 | 0 | 100% |
| backend / app / api / routes / profile.py | 43 | 2 | 0 | 95% |
| backend / app / api / routes / search.py | 156 | 10 | 0 | 94% |
| backend / app / core / config.py | 11 | 0 | 0 | 100% |
| backend / app / core / security.py | 23 | 0 | 0 | 100% |
| backend / app / db / mongo.py | 12 | 0 | 0 | 100% |
| backend / app / main.py | 13 | 1 | 0 | 92% |
| backend / app / repositories / uzytkownik_repo.py | 25 | 0 | 0 | 100% |
| backend / app / schemas / uzytkownik.py | 21 | 0 | 0 | 100% |
| backend / app / services / ai / agents.py | 123 | 13 | 0 | 89% |
| backend / app / services / ai / graph_builder.py | 16 | 0 | 0 | 100% |
| backend / app / services / ai / main.py | 46 | 2 | 0 | 96% |
| backend / app / services / ai / run_from_config.py | 36 | 4 | 0 | 89% |
| backend / app / services / ai / state.py | 61 | 0 | 0 | 100% |
| backend / app / services / ai_runner.py | 29 | 0 | 0 | 100% |
| backend / app / services / paths.py | 11 | 1 | 0 | 91% |
| backend / app / services / search_state.py | 82 | 0 | 0 | 100% |
| backend / app / tools / apply_http_tool.py | 111 | 8 | 0 | 93% |
| backend / app / tools / apply_http_wrapper.py | 47 | 1 | 0 | 98% |
| backend / app / tools / one_click_apply_tool.py | 29 | 5 | 0 | 83% |
| backend / app / tools / one_click_apply_wrapper.py | 20 | 0 | 0 | 100% |
| backend / app / tools / parser_crawler_wrapper.py | 22 | 0 | 0 | 100% |
| backend / app / tools / parser_crawler.py | 107 | 5 | 0 | 95% |
| **Total** | **1126** | **52** | **0** | **95%** |

Rysunek 2: Pokrycie testów backendowych

## 1.4 Wnioski

Przeprowadzone testy potwierdziły poprawność działania kluczowych elementów backendu i istotnej części frontendu. Szczególnie ważne było zweryfikowanie mechanizmów bezpieczeństwa,

obsługi błędnych danych wejściowych, walidacji plików CV, logiki wyszukiwania, przetwarzania danych aplikacyjnych oraz działania modułów AI i narzędzi pomocniczych.

Wszystkie uruchomione testy backendowe zakończyły się powodzeniem, co wskazuje na wysoką stabilność tej części systemu. Osiągnięte 95% pokrycia kodu backendu potwierdza, że automatyczna weryfikacja objęła zdecydowaną większość najważniejszych fragmentów aplikacji. Dodatkowo potwierdzone testy frontendowe dla warstwy HTTP/API wykazały bardzo wysoki poziom pokrycia kodu w module komunikacji z API.